

```
> = 47
read(3, "", 4096)          = 0
write(1, "jetty.x:118:69534-:/usr/share/je/... 47)jetty.x:118:69534-:/usr/
share/je/je/bin/false
) = 47
exit_group(0)              = ?
Process 16610 detached
```

A close review of the above lines will give you a lucid picture about the invoked actions. Now, we will look at how to load the module. Let's have some new code:

```
#include <linux/kernel.h>
#include <sys/syscall.h>
#include <linux/module.h>

extern void *sys_table[];
asmlinkage int (*main_sys_exit)(int);
asmlinkage int alt_exit_function(int err_code)
{
    printk("Sys_exit called with err_code=%d\n", err_code);
    return main_sys_exit(err_code);
}

int init_module()
{
    main_sys_exit=sys_table[__NR_exit];
    sys_table[__NR_exit]=alt_exit_function;
}

void cleanup_module()
{
    sys_table[__NR_exit]=main_sys_exit;
}
```

You can run:

```
gcc -Wall -DMODULE -D__KERNEL__ -DLINUX -c sample2.c
```

...to compile the code (which is assumed to be saved as *sample2.c*). Then you can use:

```
insmod filename.o // where filename is your file name
```

...to insert this module. You can use *lsmod* to list the loaded modules.

When a system function is initiated by a program, the process switches to kernel mode. In the x86 family architecture, a system call is normally initiated by the software interrupt 128 (0 80) being triggered. Now, let's take the case of the memory. In some cases, we may require dynamic memory allocation. An apt example is when you deal with temporary buffers. The functions used for this are *kmalloc()* and *kfree()*, which are implemented in the *kmalloc* file:

```
void * kmalloc (size_t size, int priority);
void kfree (void *obj);
```

```
#define kfree_s(a,b) kfree(a)
```

It is good to note that static memory allocation is also handled in a similar way. The initialisation routine for character-oriented devices for this is handled by:

```
memory_start = console_init(memory_start_memory_end);
```

For mapping functions we can have the functions in the header file *sys/mman.h*:

```
extern caddr_t mmap (caddr_t addr, size_t len,
int prot, int flags, int fd, off_t off);
extern int munmap (caddr_t addr, size_t len);
extern int mprotect (caddr_t addr, size_t len, int prot);
extern int msync;
```

An overview of the Linux kernel

We have seen some basic aspects of modules. Before we proceed, we need to go over some more general ideas about the Linux kernel. This is required for writing new modules and building your own custom kernel. In fact, there are some quick methods you can employ while making a custom package. For example, you can get the list of modules required to run during the boot process by looking at the kernel configuration (which is located in your current distribution's kernel package).

We can generally classify the Linux files as:

- Regular files
 - Directory
 - Symbolic link
 - Block-oriented device files
 - Character-oriented device files
 - Pipes (also called 'named pipes')
 - Sockets
- Beginners should note that Linux treats even directories as files. We prefer GNU/Linux OS because of its unique features. For novices, let me summarise some of the main features here:
- Multi-tasking (here processes run independent of each other)
 - Multi-user access (Linux allows a number of users to work at the same time)
 - Multi-processing (supports multi-processor architectures)
 - Architecture independence (works on a variety of hardware platforms)
 - Support for loadable executables
 - Paging *
 - Dynamic cache for hard disks
 - Shared libraries
 - Support for POSIX
 - Different formats for executable files
 - Memory protected mode
 - Support for national keyboards and fonts
 - Different file systems are supported